Q You were the i18n/l10n go-to person at OLPC; what prompted you to take a look at eBookReader?

After I became a full-time contractor for OLPC (once I had completed my graduation), my role was modified a bit—apart from i18n/l10n issues, I was also supposed to help with general software development efforts. Initially, I was looking at things like performance tuning (for example, the Browse activity being slow while drawing text, the Paint activity having redraw issues and so on), and I also took up the maintainership of a few Sugar activities.

When I was down at the OLPC office at 1CC in Cambridge, I had a few informal meetings and discussions with S J Klein about book readers (we both shared a common interest), though not much happened on a concrete basis for a few months. However, in January, when Nicholas Negroponte announced a million books as a part of OLPC's refocused mission, I was asked if I could help with sprucing up the book-reader, and I jumped in.

Q Can you tell us the technical details about what you have done?

The first job was to get a decent book-reader going in our stable builds (which is based on Fedora 9). There were quite a few issues with the Read activity that shipped with 8.2.0; for example, djvu files would make the activity crash when one tried to change the zoom level. I backported a significant part of the book-reader stack (most of the heavy duty lifting is done by Evince, btw) from our experimental Fedora (rawhide) builds, and as a result the version of the Read activity in our XO OS release 8.2.1 is much more stable. (I also managed to sneak in a few features that I had been working on.)

The next phase consisted of working closely with the upstream Sugarlabs community and tying in the development process to the Sugarlabs release schedule (with backports for OLPC-specific builds and releases, whenever required). The most important feature that was added during this phase was support for EPUB, a file format that is fast becoming the de facto standard for ebook-readers worldwide (except in the case of Amazon Kindle). EPUB uses pre-existing, well-established standards to define how a book should be laid out and formatted (for example, the content itself is usually XHTML, which can be rendered via any modern browser engine).

> "Imagine what can happen if books have a versatile tool like Etoys, Scratch or TurtleArt built in, or imagine reading through *A Byte of Python*, and being able to try out and play around with the examples within the book itself.

I also started working on annotation support. This has been tricky, since we support a variety of file formats in Read activity (PDF, PS, CBZ, CBR, DJVU, EPUB, etc), but I'm trying to have basic annotation features like bookmarking, and associating notes with the bookmarks (so a user can have at least one 'side-note' per page), etc, working for all formats.

Next versions may see format-specific annotation, and perhaps, if all goes according to plan, complete editing support for certain book formats, so that one can annotate, highlight, and do whatever one wants (of course, with the option of reverting to the older, pristine version of the book if something goes wrong).

I also added some minor usability enhancements (like a nice progress bar to show how much of the book has been read while in full screen mode, a battery icon to show the remaining charge in full-screen mode, and so on).

Of course, there is a more experimental side of my work (which I did not enable in our 'production' builds). Some of it is quite obvious, like embedding video/audio inside books, etc. However, what can be really interesting is the possibility of embedding 'creative tools' within books—tools which can be then used by the children to try out new programming languages, construct their own science simulations and experiments, and so on.

People often think of Adobe Flash when they hear this—but Flash is restrictive in the sense that the consumer is limited by the design of the original demo/simulation. Imagine what can happen if books have a versatile tool like Etoys, Scratch or TurtleArt built in, or imagine reading through *A Byte of Python*, and being able to try out and play around with the examples within the book itself (an interactive Python shell embedded in a book). I have crude proof-of-concept demos of these features lying around.

Moreover, there can be interesting stuff done to a story—a seamless transition between the narrative in a book to a movie, or even a virtual world, with characters from the book coming alive around the reader (this is one of my ideas—I don't have any code to show).

Q How does this fit within the OLPC mission and vision?

I don't think I can speak on behalf of the entire OLPC community (in fact, many will have better ideas than I